

NN-AdderNet: Nonnegative and Sparse Weight Optimization Towards Ultra-Low Bitwidth AdderNet Quantization and Compression

Yunxiang Zhang^{1*}, Gengchen Sun^{2*}, Lizhi Fang², Biao Sun², and Wenfeng Zhao¹

¹Department of Electrical and Computer Engineering, Binghamton University, Binghamton, New York, USA

²School of Electrical and Information Engineering, Tianjin University, Tianjin, China

*These authors contributed equally to this work.

sunbiao@tju.edu.cn, wzhaow@binghamton.edu

Abstract—Emerging efficient deep neural network (DNN) models, such as AdderNet, have shown great promise in significantly improving hardware efficiency compared to traditional convolutional neural networks (CNNs). However, achieving low bitwidth quantization and effective model compression remains a major challenge. To this end, we introduce Nonnegative AdderNet (NN-AdderNet), a quantization- and compression-friendly AdderNet variant that enables model compression down to 4 bits or even lower. We begin by proposing an equivalent transformation of the sum-of-absolute-difference (SAD) kernel in AdderNet, which allows for the formulation of nonnegative weights. This transformation effectively eliminates the need for a sign bit, thus saving 1 bit per weight. Next, we propose to exploit the dual-sparsity pattern in the weights of the activation-oriented NN-AdderNet quantized model. This inherent sparsity enhances the lossless compression performance over NN-AdderNet. Experimental results show that NN-AdderNet can achieve an average compressed weight bitwidth down to 4 bits or even lower, while achieving negligible accuracy loss as compared to full-precision AdderNet models. Such benefits are further illustrated with hardware-level energy and latency improvements in designing DNN inference accelerators. Consequently, the NN-AdderNet model exhibits both algorithmic and hardware efficiency, thus making it a promising candidate for resource-limited applications.

I. INTRODUCTION

Deep learning algorithms, particularly convolutional neural networks (CNNs), have achieved remarkable success in various computer vision and machine intelligence tasks [1]–[3]. However, the growing complexity of these networks, driven by the underlying deeper and wider neural network architecture and large-scale computations involving multiplication-accumulation (MAC) operations and memory accesses. To expedite computation at hardware level, current deep learning accelerators utilize massively-parallel processing elements, which has posed significant challenges for deployment on resource-constrained edge devices. [4], [5] Consequently, extensive research efforts have been dedicated to minimizing the computation and energy consumption of CNNs. Hardware-efficient CNN models, such as bottleneck architecture and point/depth-wise convolution [3], [6], [7], have been devised to amortize the computational cost. Model pruning and quantization have been widely explored [8]–[11] to reduce the MAC operations and the arithmetic/memory bitwidth, such as INT8 [12] and INT4 [13]. Nevertheless, the MAC hardware is still inevitable for the aforementioned techniques.

To address the challenge, AdderNet [14] has emerged as a promising alternative to CNNs. It replaces the costly MAC operations with hardware-efficient sum-of-absolute-difference (SAD) operations. AdderNet achieves substantial hardware

resources and power consumption reduction [15]–[21] while maintaining competitive accuracy compared to CNNs.

Despite promising, AdderNet implementation remains challenging at hardware level, especially low-bitwidth quantization and model compression [22]. Existing quantization strategies for CNNs cannot be directly applied to AdderNet due to its unique additive operation. The SAD operations in AdderNet require shared quantization scale factor between weights and activations, which complicates the quantization algorithms and hardware design. Several AdderNet-based accelerators have been demonstrated on FPGA [15]–[17] and Compute-in-Memory ASIC [20], [21] to improve the efficiency of AdderNet-based accelerators. However, these designs are mostly limited to high model quantization bitwidth (INT16), and still sub-optimal in resource utilization. WSQ-AdderNet [18] employs weight standardization technique to achieve INT8 model quantization and the collaborative packaging of parallel SAD operations in DSP-LUT hardware. Nevertheless, WSQ leads to slight model accuracy degradation. In [23], a new quantization method was introduced by employing group-shared scaling and dynamic clipping to handle weights and activations. By clustering weights and applying custom clipping techniques, this approach indeed improves the performance of low-bit AdderNet models, however, it substantially increases the computational and hardware implementation complexities. Alternatively, an activation-oriented quantization (AOQ) technique was proposed [19], aiming at reducing the bitwidth of weights and activations to INT6 with minimal model accuracy degradation. A fused-bias removal strategy was also introduced to minimize on-chip memory requirements by eliminating redundant bias accumulation.

Despite the consistent efforts, AdderNet models are still limited to INT6 quantization. Moreover, the signed Laplacian distribution of weights render it challenging to achieve high compression ratio using lossless entropy coding scheme. To further optimize AdderNet quantization and compression, we propose a novel nonnegative AdderNet (NN-AdderNet) framework. NN-AdderNet further incorporates sparse weight optimization based on a lossless compression scheme, enabling substantial improvement in model compression performances and ultimately can be exemplified with accelerator hardware level benefits. Specifically, we reformulate the signed SAD of AdderNet into nonnegative SAD operations. Combined with activation-oriented quantization, this results in a novel dual-sparsity weight distribution in NN-AdderNet. When incorporating Huffman encoding, our approach ensures sub-INT4

quantization across a wide collection of AdderNet-based DNN models. The main contributions are summarized as follows:

- We introduce a novel nonnegative AdderNet formulation that leads to unsigned SAD kernel, thereby facilitating model quantization with sign-bit savings;
- We propose an activation-oriented nonnegative AdderNet quantization that can promote a novel dual-sparsity in the nonnegative weights, which ultimately enhances the entropy coder performance;
- We demonstrate substantial model compression benefits of NN-AdderNet, with $1.5\sim 1.67\times$ model size reduction on uncompressed 8-bit models, thereby leading to sub-4bit compressed models with merely $0.3\%\sim 1.6\%$ model accuracy drop;
- We show significant NN-AdderNet accelerator latency improvement of up to $34.8\%\sim 38.6\%$ over CNN accelerators for sub-4bit compressed models, and $13\%\sim 16.2\%$ latency improvement for sub-3bit compressed models.

This paper is organized as follows. Section II introduces the background and motivation. Section III details proposed Nonnegative AdderNet. Section IV uncovers the dual-sparsity profiles in the NN-AdderNet weights. Section V presents the experimental results. Section VI concludes the paper.

II. BACKGROUND AND MOTIVATION

A. Preliminaries on AdderNet

In a conventional CNN model, for a given input feature map $X \in \mathbb{R}^{h \times w \times c_{in}}$ and weight $W \in \mathbb{R}^{k \times k \times c_{in} \times c_{out}}$, the convolutional operation is defined as:

$$Y(m, n, c_{out}) = \sum_{i=1}^k \sum_{j=1}^k \sum_{l=1}^{c_{in}} X(m+i, n+j, l) \times W(i, j, l, c_{in}), \quad (1)$$

where h and w are the height and width of the input, $k \times k$ is the size of the filter kernel, c_{in} and c_{out} are the number of input and output channels, m and n are the height and width of the output, respectively. In AdderNet [14], convolution operations are replaced by SAD operations, [24]–[26] aiming to reduce the computational overhead of the expensive MAC operations:

$$Y(m, n, c_{out}) = - \sum_{i=1}^k \sum_{j=1}^k \sum_{l=1}^{c_{in}} |X(m+i, n+j, l) - W(i, j, l, c_{in})|, \quad (2)$$

where $|\cdot|$ is the absolute function, which involves only subtraction (addition equivalent), multiplexer, and negate. In the following part of the paper, we will simplify the subscripts and write the SAD operation in a more compact form as

$$Y = - \sum |X - W|. \quad (3)$$

B. AdderNet Quantization Challenge

A significant number of early-stage AdderNet quantization studies are limited to INT16. This limitation arises as AdderNet presents fundamentally different quantization requirements compared to convolutional kernels. Existing CNN model quantization techniques are not directly applicable to AdderNet. In particular, the exchange law does not hold in the

SAD operation, meaning that independent scaling factors for weights (s_w) and activations (s_x) cannot be used for AdderNet quantization, as is typically done in CNNs:

$$\text{SAD}(s_x X, s_w W) \neq (s_x s_w) \times \text{SAD}(X, W). \quad (4)$$

Therefore, AdderNet quantization requires the use of a shared scale s_{common} to quantize weights and activations. So, quantization and dequantization weights are defined as:

$$\text{SAD}(sX, sW) = s_{\text{common}} \times \text{SAD}(X, W) \quad (5)$$

Hence, due to the limitation of the shared scale factor during model quantization in AdderNet, existing quantization techniques do not perform well, which results in a higher bitwidth (≥ 10 bits) of the SAD kernel compared to the convolutional kernel (6 bits) for the same inference accuracy. As a result, the potential hardware gains of AdderNet may diminish. The source of quantization error in AdderNet is mainly caused by the value difference between the weights and the input activations. The weights in AdderNet obey the Laplace distribution with a mean of zero, but the variance is much larger than Activations.

One approach to improve weight quantization is to reduce the weight dynamic range, as demonstrated in [18], as the dynamic range or distributions of the ReLU activations can barely be easily altered. WSQ-AdderNet [18], which employs weight normalization (WS) technique reduces the weight dynamic range by approximately a factor of 4, thereby achieving INT8 weight quantization. Nevertheless, this normalization results in a decrease in model accuracy even in full-precision training, with a drop of about 1% compared to baseline AdderNet. To achieve model quantization below INT8 while retaining model accuracy, AdderNet 2.0 [19] introduces an AOQ technique to achieves INT6 model quantization. This is accomplished by quantizing activations to 5 bits, while the weights are clipped to the range of -32 to 31, resulting in an effective 6-bit weight quantization. To ensure correct SAD operation, a weight bias is generated during weight clipping process, and the significant weight bias is handled by either adding it back to the feature map or fusing it into the batch normalization bias, as it represents a portion of the mean offset. However, further reducing the quantization bitwidth leads to increased activation quantization errors, causing a sharp drop in model accuracy.

III. PROPOSED NONNEGATIVE ADDERNET

In this section, we detail the mathematical foundations of nonnegative AdderNet.

A. Equivalent Transformation of SAD to Nonnegative SAD

Theorem 1. *Given an SAD operation with signed weights W , which has been trained to achieve high performance on a training set, and access to a representative subset of the input data, there exists a family of equivalent transformations that convert the signed SAD operation to a nonnegative SAD*

operation with nonnegative weights W_+ , without any loss in performance. One typical transformation is:

$$\sum |X - W| = \sum |X - \text{ReLU}(W)| + \sum |W_-|, \quad (6)$$

where W_- represents negative weights and ReLU denotes the rectified linear unit function. $\sum |W_-|$ represents the sum of the absolute values of all negative weights, which can be precomputed without incurring extra computation after model training.

Proof. In AdderNet, each layer's activation function is ReLU, meaning that the activations X are always nonnegative, whereas the weights W in the original AdderNet can be signed, taking positive, zero, or negative values. To better illustrate the relationships between X and W under different conditions, we can simplify the scenario by assuming that both X and W correspond to points on a sine wave, as shown in Fig. 1. The figure is divided into five regions, each representing a different relationship between X and W , including both magnitude and sign variations. These five regions encompass all possible relationships between X and W . For example, region 1 represents the case where both X and W are positive, with W being greater than X , while region 3 represents the case where X is positive and W is negative, in which case it is clear that X is greater than W . Based on the above regional division, an SAD operation can be decomposed into the summation of the results from the five regions as:

$$\begin{aligned} \sum |X - W| &= \sum_1 |X_1 - W_1| \\ &\quad + \sum_2 |X_2 - W_2| + \sum_3 |X_3 - W_3| \\ &\quad + \sum_4 |X_4 - W_4| + \sum_5 |X_5 - W_5| \\ &= \sum_1 W_1 - \sum_1 X_1 + \sum_2 X_2 - \sum_2 W_2 \\ &\quad + \sum_3 X_3 - \sum_3 W_3 + \sum_4 W_4 - \sum_5 W_5 \\ &= \sum_1 W_1 - \sum_1 X_1 + \sum_2 X_2 - \sum_2 W_2 + \sum_4 W_4 \\ &\quad + \sum_3 X_3 \\ &\quad + \left(-\sum_3 W_3 - \sum_5 W_5 \right) \\ &= \sum_{1,2,4} |X_{1,2,4} - W_{1,2,4}| + \sum_{3,5} |X_{3,5} - 0| \\ &\quad + \sum_{3,5} |W_{3,5}| \\ &= \sum |X_+ - W_+| + \sum |X_- - 0| + \sum |W_-| \\ &= \sum |X - \text{ReLU}(W)| + \sum |W_-|, \end{aligned} \quad (7)$$

where the subscript of the summation symbol indicates the region over which the summation is performed. W_+ and W_- represent the positive and negative weights, while X_+ and X_- are the corresponding activations, respectively. The $+$ under the summation symbol indicates that the summation is

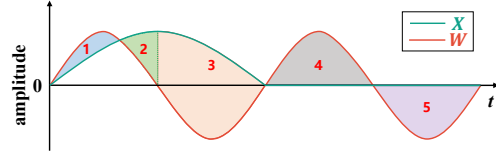


Fig. 1. Illustration of X and W with varying signs and amplitudes. The numbers in the figure represent the regions when X and W take different signs and amplitudes.

performed over the region where W is positive, while the $-$ under the summation symbol indicates that the summation is performed over the region where W is negative. $\square$

Remarks. (i) The theorem mentions a family of transformations since the transformation is not unique, that is, different nonnegative AdderNet formulations give rise to an exact transformation. In this paper, we propose the transformation in Eq. (6) because it is most conducive to quantization. Although other literature has introduced various transformations applicable to different scenarios, they are not suitable for either quantization or excessive computation [21]. (ii) A consequence of the exact transformation is that both signed AdderNet and nonnegative AdderNet have exactly the same performance on a sample-by-sample basis: if for a specific sample the prediction of the signed AdderNet is wrong then this is also the case for the nonnegative AdderNet, and vice versa.

B. Activation-Oriented Quantization for NN-AdderNet

For a q -bit quantization in Adder layer, the quantized weights can be obtained from the original floating-point weights based on a quantization step size, which typically results in a bitwidth much larger than q . Subsequently, a weight clipping process [19] yields clipped weights W_{clip} with a bitwidth of q .

$$W_{\text{clip}}(w) = \begin{cases} -2^{q-1}, & w < -2^{q-1} \\ w, & w \in [-2^{q-1}, 2^{q-1} - 1] \\ 2^{q-1} - 1, & w > 2^{q-1} - 1 \end{cases} \quad (8)$$

The difference between the original weights W and the clipped weights W_{clip} is defined as the weight bias W_{bias} .

$$W_{\text{bias}} = -|W - W_{\text{clip}}|, \quad (9)$$

Hence, the SAD operations outlined in Eq. 3 can be reformulated as:

$$-\sum |X - W| = -\sum |X - W_{\text{clip}}| + \sum W_{\text{bias}}, \quad (10)$$

where $\sum |X - W_{\text{clip}}|$ represents q -bit clipped SAD operations, and $\sum W_{\text{bias}}$ is a constant after model training and can be pre-computed without incurring extra hardware overheads.

For a ReLU-activation based AdderNet model, the computation to obtain one output feature pixel is the SAD operations between the input features and the weight kernels across all input channels. After model training, both positive and negative weights are fixed, leaving only the feature maps changing with respect to different inputs. Based on Theorem

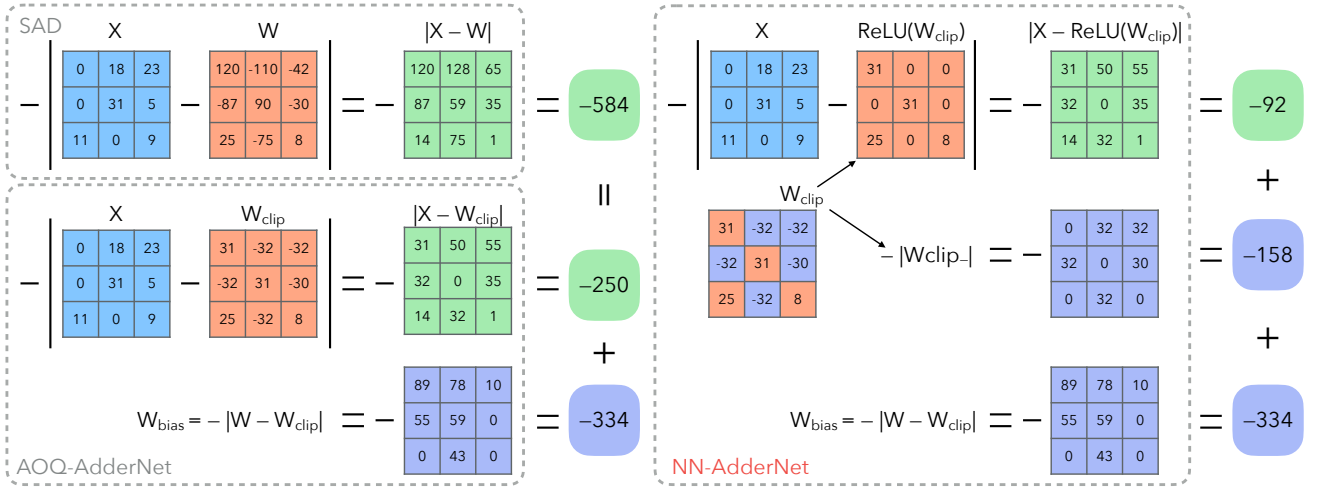


Fig. 2. A conceptual illustration of basic SAD operations in AdderNet (top left), AOQ with weight clipping (bottom left) in AOQ-AdderNet, and the proposed NN-AdderNet (right).

1, Eq. (6) holds valid with the assumption of ReLU activations, which is the widely adopted activation function used in popular DNN models like AdderNet. Obviously, Eq. (6) suggests that the basic SAD computation with signed weights is mathematically equivalent to the unsigned SAD computation with ReLU weights plus the sum of the absolute of all negative weights $\sum |W_-|$. Note that $\sum |W_-|$ can be pre-computed after model training and ultimately handled by fuse-bias removal technique, which will not incur additional computation overheads during model inference. Hence, Eq. (6) demonstrates the feasibility of NN-AdderNet, where negative weights contribution to the AdderNet model computation will be all fused to a *feature bias* that can be removed by BN, leaving only positive weights matter to the computation. Moreover, Eq. (6) holds valid for both floating-point and quantized operations, which can be seamlessly applied with the activation-oriented quantization scheme.

Hence, combining Eq. (6) and Eq. (10), the SAD operations in AdderNet can be computed as:

$$-\sum |X - ReLU(W_{clip})| + \sum W_{bias} - \sum |W_{clip-}|, \quad (11)$$

where the term $(\sum W_{bias} - \sum |W_{clip-}|)$ is a constant after model training, and can be handled by fused bias removal technique, leaving the first term $-\sum |X - ReLU(W_{clip})|$ performing the essential quantized NN-AdderNet operation. A conceptual illustration of the vanilla SAD of AdderNet, Activation-oriented quantization of AOQ-AdderNet, and the proposed Activation-oriented NN-AdderNet have been demonstrated in Fig. 2.

IV. DUAL-SPARSITY PROFILES IN NN-ADDERNET

In this section, we present details on NN-AdderNet to achieve improved quantization and compression performance.

Lossless compression of weights using entropy coding scheme, such as Huffman coding or Run Length Encoding (RLE), achieves limited compression ratio due to Gaussian- or Laplacian-like signed weight distributions in CNN and

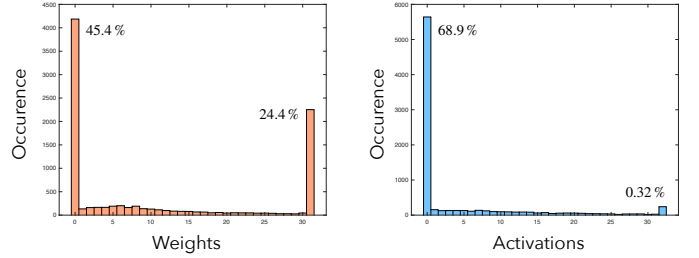


Fig. 3. A representative weight and activation distributions in quantized NN-AdderNet. For weights, a dual-sparsity profile can be observed, while activations exhibit the nominal '0' sparsity.

AdderNet, respectively. Basically, entropy coders rely on the knowledge of the probability of the source symbols to construct optimal coding for high compression ratio. Taking Huffman coding [27] as an example, this lossless compression algorithm constructs an optimal prefix code by analyzing the frequency of symbols in the source data, thereby reducing the total number of bits required to represent the data. Hence, to improve the compression performance of Huffman coder, the key is to skew the probability of symbols. Equivalently, the sparsity of certain symbols should be enhanced.

As we have already demonstrated in Eq. (11), vanilla AdderNet execution can be equivalently achieved via NN-AdderNet. In this way, we consider the lossless compression of quantized NN-AdderNet, i.e., performing Huffman encoding over the $ReLU(W_{clip})$. Fig. 3 shows the typical weight and activation distributions of NN-AdderNet after UINT5 activation-oriented quantization. As can be observed, after the nonnegative ReLU and weight clipping operations during the quantization process, the quantized weights of NN-AdderNet exhibit a unique distribution profile. Unlike previous approximate symmetrical signed weight distributions, the weights distribution of NN-AdderNet show a strong dual-sparsity profile, where a great portion of '0' symbols (45.4%, caused by ReLU operation), and significantly increased amount of '31' symbols (24.4%,

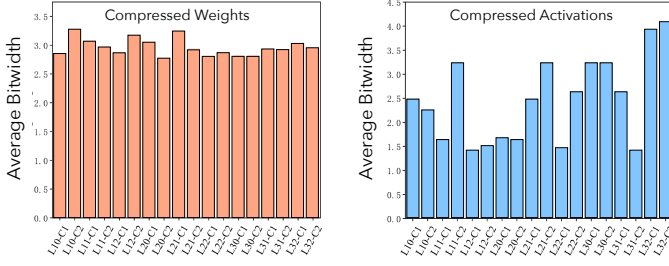


Fig. 4. ResNet-20 AdderNet on CIFAR-10 dataset: cross-layer compressed bitwidth of weights (left) and activations (right), respectively.

mainly from weight clipping) co-exist. It becomes apparent that the weight sparsity in the quantized NN-AdderNet has been significantly promoted, which will ultimately lead to improved compression performance of lossless entropy coders like Huffman coder. Moreover, the UINT5 activations are mainly from quantized ReLU operation, hence a more pronounced portion of ‘0’ symbol (68.9%) is observed. Hence, ReLU activations can also be efficiently handled by entropy coders to achieve bitwidth reduction.

Fig. 4 shows the averaged number of compressed bits required for the weights $\text{ReLU}(W_{\text{clip}})$ and ReLU activations across Adder layers in ResNet20, respectively. Due to the unique dual-sparsity feature in quantized weights, it becomes possible for all the weights in the Adder layers to be compressed down to UINT3 (2.97-bit on average). For ReLU activations, due to a varying probability of ‘0’s, the average compressed bits across layers also vary significantly (1.42 ~ 3.25-bit).

V. EXPERIMENTAL RESULTS

A. DNN Model Preparation & Setup

To evaluate the performance of the proposed NN-AdderNet, we conducted experiments using the ResNet20 [2], ResNet32 [2], and ResNet50 [2] backbones across three datasets: CIFAR-10, CIFAR-100 [28], and ImageNet, respectively. The ResNet20-based NN-AdderNet was trained from scratch on the CIFAR-10 dataset. The training setup utilized a GeForce RTX 3090 GPU with CUDA 12.6, Python 3.9, and PyTorch 2.0, running on an Ubuntu 22.04 LTS operating system. The training configuration included an initial learning rate of 0.05, a batch size of 256, and the SGD optimizer with a momentum of 0.9 and a weight decay of 5×10^{-4} . Additionally, we applied an adaptive learning strategy with cosine learning rate decay, as detailed in [29]. The ResNet32-based NN-AdderNet was also trained from scratch, but on the CIFAR-100 dataset, using the same setup as the ResNet20 experiment. The learning rate, batch size, and optimizer settings were kept consistent to ensure comparability. For ResNet50-based NN-AdderNet, we initially used a pre-trained model on the ImageNet dataset [14]. We then applied the proposed non-negative quantization technique, which involves nonnegative weight transformation and quantization, to obtain the final NN-AdderNet models. We consider uncompressed 8-bit and 5-bit

TABLE I
SUMMARY OF ACCELERATOR ARCHITECTURAL SPECIFICATIONS

Architecture	PE	DRAM BW	SRAM Buffer
ResNet20/32	16×16	64-bit	100-KB
ResNet50	32×32	256-bit	10-MB

post-training quantization for all DNN models, including the original CNN, AOQ-AdderNet, and NN-AdderNet to highlight the advantages and limitations of the proposed method. For uncompressed 8-bit models, quantization bitwidth of INT8, INT8, and UINT8 were adopted for CNN, AOQ-AdderNet, and NN-AdderNet weights, respectively. For uncompressed 5-bit models, CNN and NN-AdderNet adopts INT5 and UINT5 quantization bitwidth, respectively.

B. Hardware Validation Setup

For hardware-level evaluation, including energy and processing latency, we resort to CNNergy [30], an analytical simulator model for CNN performance estimation and optimization. Essential modifications were made to CNNergy simulator to support AdderNet evaluation. For the sake of fairness, register-transfer-level (RTL) designs for the multiply-accumulate (MAC) and the sum-of-absolute-difference (SAD) with varied bitwidth were implemented in Verilog HDL. The RTL designs were synthesized using Synopsys Design Compiler using a commercial 65-nm low-power CMOS process, all with a peak operating frequency of 200-MHz at nominal supply voltage 1.2-V and room temperature of 27°C. The synthesized results were then normalized to 45nm and used as the inputs of the CNNergy simulator.

The accelerator architectural specifications are summarized in Table I. The granularities of the processing elements (PEs) were determined with respect to different DNN architectures, where 256 and 1024 PEs were adopted for ResNet20/32 and ResNet50 architectures, respectively. The total off-chip communication bandwidth to the DRAM and the on-chip SRAM buffer size were set to be 64-bit and 256-bit, 100-KB and 10-MB for ResNet20/32 and ResNet50 architectures, respectively. The weights and activation compression were benchmarked directly using the test datasets across CIFAR-10/100 and ImageNet, which will be used to accurately assess both the energy and latency benefits obtained from the communication bandwidth reduction between the off-chip DRAM and on-chip global buffer. Huffman codec was used for lossless data compression and decompression.

C. Experimental Result

1) *Model Compression Performance*: Fig. 5 shown the experimental results on weight and activation lossless compression performance of CNN, AOQ-AdderNet, and NN-AdderNet across different model architectures, datasets, quantization levels, and the respective quantized model accuracies.

For model quantization and compression, NN-AdderNet possesses greater weight compression potential due to its unique dual-sparsity characteristics as compared to CNN and

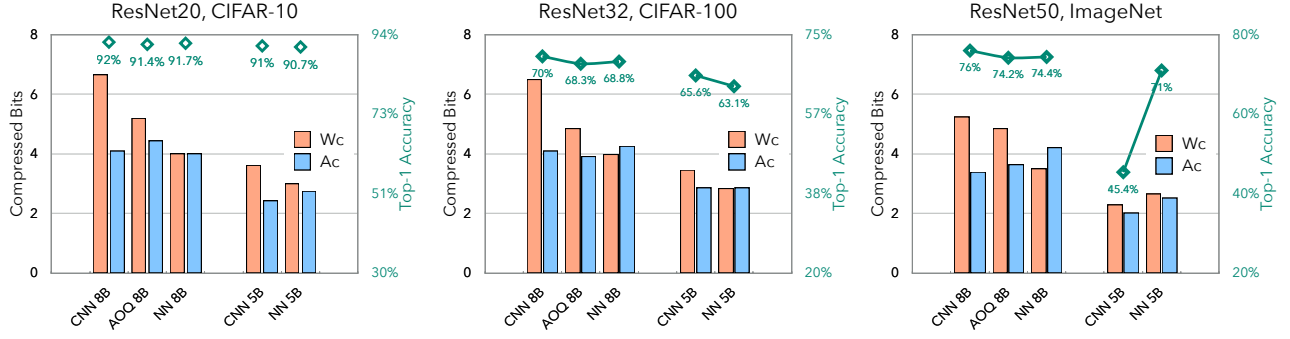


Fig. 5. The lossless compressed weights and activations (averaged) of the original 8-bit and 5-bit CNN, AOQ-AdderNet, and NN-AdderNet models. The corresponding model accuracies are also annotated.

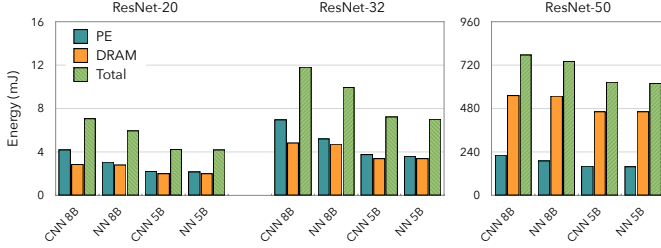


Fig. 6. Energy consumption (lossless compressed) of 8-bit and 5-bit CNN and NN-AdderNet models across ResNet20, ResNet32, and ResNet50.

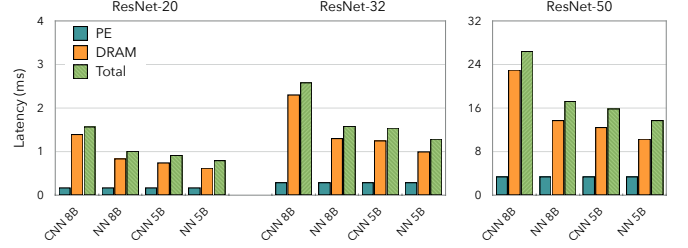


Fig. 7. Inference latency (lossless compressed) of 8-bit and 5-bit CNN and NN-AdderNet models across ResNet20, ResNet32, and ResNet50.

AOQ-AdderNet that only have signed weights with low sparsity. As illustrated in Fig. 5, for 8-bit models, NN-AdderNet achieves 2.65-bit, 2.52-bit, and 1.74-bit savings on ResNet20, ResNet32, ResNet50 as compared to the CNN baseline. On average, the compressed weights achieve UINT4 or lower averaged compressed bits, as opposed to 5.2 $\sim$ 6.6-bit in CNN models. The corresponding model accuracy drop due to employing NN-AdderNet is only 0.3%, 1.2%, and 1.6% on ResNet20, ResNet32, ResNet50, respectively. For 5-bit models, NN-AdderNet achieves 0.61-bit (ResNet20) and 0.63-bit (ResNet32) savings as compared to the CNN baseline. On average, the compressed weights achieve UINT3 or lower averaged compressed bits, as opposed to 3.6-bit for compressed CNN weights. The corresponding model accuracy drop of NN-AdderNet is 0.3% and 2.5% on ResNet20 and ResNet32, respectively.

2) *Hardware-Level Demonstration:* For compressed ReLU activations illustrated in Fig. 5, NN-AdderNet exhibits comparable but slightly lower sparsity compared to its CNN counterpart. While this won't pose a significant challenge from a model compression standpoint, compressed activations can still impact the overall accelerator energy consumption and latency during inference.

The energy consumption of different DNN accelerators are illustrated in Fig. 6. While PE energy is higher than the off-chip communication to DRAM energy for both CNN and NN-AdderNet on ResNet20/32, off-chip communication to DRAM energy becomes dominant for larger models like ResNet50. The total energy savings for 8-bit NN-AdderNet can be up to

16% over the CNN counterpart, whereas the energy benefits become marginal for 5-bit models.

The inference latency of the NN-AdderNet accelerator are clearly advantageous, mainly originated from the significantly reduced off-chip communication to DRAM. As shown in Fig. 7, the PE latency remains constant for a given model architecture, as all models were assumed to operate at 200-MHz. As expected, due to higher attainable compression ratio of the NN-AdderNet model, 8-b NN-AdderNet model achieves 35.7%, 38.6%, and 34.8% total latency reduction, while 5-bit NN-AdderNet model achieves 13%, 16.2%, and 13.5% total latency reduction, all compared with their CNN counterparts.

VI. CONCLUSION

In this paper, we present a novel quantization- and compression-friendly nonnegative AdderNet, which is a mathematically equivalent formulation of vanilla AdderNet with signed weights. The nonnegative formulation, together with the activation-oriented quantization scheme, yields ultra-low bitwidth AdderNet quantization and compression. Experimental results demonstrate that NN-AdderNet achieves unsigned integer quantization, and further promote a novel dual-sparsity distribution in NN-AdderNet weights. Hence, model compression of sub-4bit becomes possible. Experimental results show uncompressed 8-bit and 5-bit NN-AdderNet can achieve sub-UINT4 and sub-UINT3 bitwidth, respectively. Correspondingly, this leads to up to 36% latency and 16% energy improvement of NN-AdderNet based accelerators over the CNN counterpart. These results demonstrate that NN-AdderNet can be advantageous in both algorithm and hardware efficiency.

REFERENCES

- [1] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *nature*, 521(7553):436–444, 2015.
- [2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [3] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.
- [4] Vivienne Sze, Yu-Hsin Chen, Tien-Ju Yang, and Joel S Emer. Efficient processing of deep neural networks: A tutorial and survey. *Proceedings of the IEEE*, 105(12):2295–2329, 2017.
- [5] Zhi Tian, Xiangxiang Chu, Xiaoming Wang, Xiaolin Wei, and Chunhua Shen. Fully convolutional one-stage 3d object detection on lidar range images. *Advances in Neural Information Processing Systems*, 35:34899–34911, 2022.
- [6] Forrest N Iandola, Song Han, Matthew W Moskewicz, Khalid Ashraf, William J Dally, and Kurt Keutzer. Squeezenet: Alexnet-level accuracy with 50x fewer parameters and 0.5 mb model size. *arXiv preprint arXiv:1602.07360*, 2016.
- [7] Haoran You, Xiaohan Chen, Yongan Zhang, Chaojian Li, Sicheng Li, Zihao Liu, Zhangyang Wang, and Yingyan Lin. Shiftaddnet: A hardware-inspired deep network. *Advances in Neural Information Processing Systems*, 33:2771–2783, 2020.
- [8] Song Han, Huizi Mao, and William J Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. *arXiv preprint arXiv:1510.00149*, 2015.
- [9] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2704–2713, 2018.
- [10] Mengshu Sun, Zhengang Li, Alec Lu, Yanyu Li, Sung-En Chang, Xiaolong Ma, Xue Lin, and Zhenman Fang. Film-qnn: Efficient fpga acceleration of deep neural networks with intra-layer, mixed-precision quantization. In *Proceedings of the 2022 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, pages 134–145, 2022.
- [11] Junsong Wang, Qiuwen Lou, Xiaofan Zhang, Chao Zhu, Yonghua Lin, and Deming Chen. Design flow of accelerating hybrid extremely low bit-width neural network in embedded fpga. In *2018 28th international conference on field programmable logic and applications (FPL)*, pages 163–169. IEEE, 2018.
- [12] Xilinx. Wp486: Deep learning with int8 optimization on xilinx devices. *White Paper*, 2017.
- [13] AmirAli Abdolrashidi, Lisa Wang, Shivani Agrawal, Jonathan Malmaud, Oleg Rybakov, Chas Lechner, and Lukasz Lew. Pareto-optimal quantized resnet is mostly 4-bit. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3091–3099, 2021.
- [14] Hanqing Chen, Yunhe Wang, Chunjing Xu, Boxin Shi, Chao Xu, Qi Tian, and Chang Xu. Addernet: Do we really need multiplications in deep learning? In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 1468–1477, 2020.
- [15] Yunhe Wang, Mingqiang Huang, Kai Han, Hanqing Chen, Wei Zhang, Chunjing Xu, and Dacheng Tao. Addernet and its minimalist hardware design for energy-efficient artificial intelligence. *arXiv preprint arXiv:2101.10015*, 2021.
- [16] Mingyong Zhuang, Xinhui Liao, Huhong Wu, Jianyang Zhou, and Zichao Guo. Vlsi architecture design for adder convolution neural network accelerator. In *2021 IEEE 15th International Conference on Anti-counterfeiting, Security, and Identification (ASID)*, pages 5–9. IEEE, 2021.
- [17] Ke Ma, Lei Mao, and Shinji Kimura. A novel fpga-based convolution accelerator for addernet.
- [18] Yunxiang Zhang, Biao Sun, Weixiong Jiang, Yajun Ha, Miao Hu, and Wenfeng Zhao. Wsq-addernet: Efficient weight standardization based quantized addernet fpga accelerator design with high-density int8 dsp-lut co-packing optimization. In *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*, pages 1–9, 2022.
- [19] Yunxiang Zhang, Omar Al Kailani, and Wenfeng Zhao. Addernet 2.0: Optimal fpga acceleration of addernet with activation-oriented quantization and fused bias removal based memory optimization. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*, pages 1–6, 2024.
- [20] Soyeon Um, Sangyeob Kim, Sangjin Kim, and Hoi-Jun Yoo. A 43.1 tops/w energy-efficient absolute-difference-accumulation operation computing-in-memory with computation reuse. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 68(5):1605–1609, 2021.
- [21] Haikang Diao, Yifan He, Xuan Li, Chen Tang, Wenbin Jia, Jinshan Yue, Haoyang Luo, Jiahao Song, Xueqing Li, Huazhong Yang, et al. A multiply-less approximate sram compute-in-memory macro for neural-network inference. *IEEE Journal of Solid-State Circuits*, 2024.
- [22] Zechun Liu, Zhiqiang Shen, Marios Savvides, and Kwang-Ting Cheng. Reactnet: Towards precise binary neural network with generalized activation functions. In *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XIV 16*, pages 143–159. Springer, 2020.
- [23] Ying Nie, Kai Han, Haikang Diao, Chuanjian Liu, Enhua Wu, and Yunhe Wang. Redistribution of weights and activations for addernet quantization. *Advances in Neural Information Processing Systems*, 35:22739–22751, 2022.
- [24] Xinghao Chen, Chang Xu, Minjing Dong, Chunjing Xu, and Yunhe Wang. An empirical study of adder neural networks for object detection. *Advances in Neural Information Processing Systems*, 34:6894–6905, 2021.
- [25] Dehua Song, Yunhe Wang, Hanqing Chen, Chang Xu, Chunjing Xu, and DaCheng Tao. Addersr: Towards energy efficient image super-resolution. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 15648–15657, 2021.
- [26] Bowen Li, Yao Xin, Chao Li, Youneng Bao, Fanyang Meng, and Yongsheng Liang. Adderlc: towards low computation cost image compression. In *ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 2030–2034. IEEE, 2022.
- [27] David A Huffman. A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, 40(9):1098–1101, 1952.
- [28] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009.
- [29] Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts. *arXiv preprint arXiv:1608.03983*, 2016.
- [30] Susmita Dey Manasi, Farhana Sharmin Snigdha, and Sachin S Sapatnekar. Neupart: Using analytical models to drive energy-efficient partitioning of cnn computations on cloud-connected mobile clients. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 28(8):1844–1857, 2020.